start. Additionally, the framework includes user-friendly helper functions that facilitate expressive application testing.

For testing Laravel applications, alternative tools like PEST can also be employed.

Within the tests directory of your project, you will find two automatically generated folders: Feature and Unit. Unit tests focus on isolated sections of your code, often targeting a single method. These tests in the **`Unit`** directory operate independently of your Laravel application's database or other framework services, as they do not boot the application.

On the other hand, Feature tests cover more extensive aspects of your code, including interactions between multiple objects or complete HTTP requests to JSON endpoints. They should constitute the bulk of your testing efforts, offering the highest level of assurance that your system is functioning as intended.

Both the Unit test and Feature test folders contain an **`ExampleTest.php`** file. Following the installation of a new Laravel application, you can execute your tests using either the **`vendor/bin/phpunit`** command or the **`php artisan test`** command.

Setting up Environment for Testing

Laravel autonomously switches the configuration environment to **`testing`** when tests are executed due to the environment parameters specified within the **`phpunit.xml`** file. Moreover, during the testing phase, Laravel configures the session and cache to the **`array`** driver by default, ensuring no persistence of session or cache data.

If you find the need for additional configuration parameters within the **`testing`** environment, you have the option to create them. You can establish these testing environment variables within your application's **`phpunit.xml`** file. However, before running your tests, it's crucial to execute the **`config:clear`** Artisan command in order to clear the configuration cache.

`.env.testing` Environment File

A **`.env.testing`** file can also be created and placed in the project root. When running PHPUnit tests or using the **`--env=testing`** option when executing Artisan commands, this file will be utilized in place of the **`.env`** file.